have to quote everything.

- Words and file names are not expanded. However, other forms of expansion such as text expansions and substitutions are performed.
- The = operator behaves like the way = or == operators do with *[*.
- The != and == operators compare the text expression on the left with a pattern on the right.
  - A pattern is a text string containing at least one wildcard character (* or ?) or a bracket expression [..]. A bracket expression encloses a set of characters or a range of characters (separated by a hyphen (-)) between the square brackets (*[* and *]*).
- A new =~ operator is available. (It cannot be used with *[*.) It compares the text expression on the left with a regular expression on the right. (It will exit with a return value of 2 if the regular expression is *invalid*.) The =~ operator is great for matching substrings.

```
# Matches substring ell
$ if [[ “Hello?” =~ ell ]]; then echo
“Yes”; else echo “No”; fi
Yes

# Matches substring Hell at beginning
$ if [[ “Hello?” =~ ^Hell ]]; then echo
“Yes”; else echo “No”; fi
Yes

# Does not match substring ? (a regex
special character) at the end
$ if [[ “Hello?” =~ ?$ ]]; then echo
“Yes”; else echo “No”; fi
No

# Matches substring ? at the end when
quoted
$ if [[ “Hello?” =~ “?”$ ]]; then echo
“Yes”; else echo “No”; fi
Yes
```

Evaluations with *[* and *[[* have their legitimate use cases. Do not use one for the other (se Table below).

Do not use *-a* and *-o* logical operators. You will make mistakes reading and writing them. They are the *sh* way of doing things. Square brackets and operators && and || are so bash.

Arithmetic operations are not straightforward

If you set *a=1* and then try *a=a+1*, does *$a* echo as 2 or 11? The answer is *a+1*. Until a few years back, I did not know how to perform arithmetic operations in bash. I never had to so I never learned it. I just assumed that it must be the same as in other languages but it was not to be. To add one plus one, you can use:

```
let a=a+1

# or


a=$(( a+1 ))
```

Array operations can be cryptic

Does every language out there need to have a totally different method to create and use arrays? Who is so evil? Why?

Operator use	Result
[-f file]	Does it exist as a file?
[-d file]	Does it exist as a directory?
[-h file]	Does it exist as a soft link?
[-r file]	Is the file readable?
[-w file]	Is the file writeable?
[-z file]	Is the string empty?
[-n file]	Is the string not empty?
[string1 = string2]	Are the strings same? = is same as ==
[string1 != string2]	Are the strings different?
[n1 -eq n2]	Are the numbers same?
[n1 -ne n2]	Are the numbers different?
[n1 -le n2]	Is n1 less than or equal to n2?
[n1 -ge n2]	Is n1 greater than or equal to n2?
[n1 -lt n2]	Is n1 less than n2
[n1 -gt n2]	Is n1 greater than n2
[! e]	Is the expression *false?*
[e1] && [e2]	Are both expressions *true?*
[e1] || [e2]	Is one of the expressions *true?*
[[string1 = string2]]	Are the strings same? Behaves like == in single-square-bracket evaluations.
[[string1 == string2]]	Does *string1* match the pattern *string2?*
[[string1 != string2]]	Does *string1* not match the pattern *string2?*
[[string1 =~ string2]]	Does *string1* not match the regular expression *string2?*